

LU3IN025 – Intelligence Artificielle et Jeux

Projet 3 : Gale–Shapley et PLNE

Nehir Yükksekaya

Kathy Vo

Année 2025–2026

Cadre du projet

On considère le problème d’affectation des $n = 13$ étudiants $Etu0, \dots, Etu12$ dans les $m = 10$ parcours du Master Informatique (*AI2D*, *BIM*, *CCA*, *IMA*, *MIND*, *QI*, *RES*, *SAR*, *SESI*, *STL*). Les capacités fournies par `PrefSpe.txt` sont

$$\text{cap} = (2, 1, 1, 1, 2, 1, 1, 1, 1, 2), \quad \sum_j \text{cap}_j = 13 = n,$$

ce qui rend possible un mariage parfait (chaque étudiant affecté à exactement un parcours).

Convention de Borda. Pour un classement de m alternatives, la k -ième préférée a pour utilité $m - k + 1$. Avec un rang commençant à 0 dans le code :

$$u_E[i, j] = m - \text{rang}_E[i, j], \quad u_P[j, i] = n - \text{rang}_P[j, i].$$

Le 1^{er} choix de chaque étudiant vaut donc $u = m = 10$, son dernier vaut $u = 1$. Cette convention rend l’indication donnée en Q13 cohérente : « i est dans ses k premiers choix $\iff u_E[i, \cdot] \geq m - k + 1 \geq m - k$ ».

1 Algorithmes de Gale–Shapley (Q1–Q6)

Q1 – Lecture des fichiers

Les fonctions `lireCE` et `lireCP` (`exemple.py`) construisent respectivement :

- C_E : matrice $n \times m$ telle que $C_E[i]$ est la liste des parcours rangés par préférence décroissante de l’étudiant i ;
- C_P : matrice $m \times n$ analogue pour les parcours, ainsi que le vecteur des capacités `cap`.

Q2 – Structures de données pour Gale–Shapley côté étudiants

L’algorithme exécute trois opérations critiques. Pour qu’elles soient en $\mathcal{O}(1)$ amorti, on utilise les structures suivantes.

Opération	Structure	Coût
Trouver un étudiant libre	file <code>libre</code> (FIFO)	$\mathcal{O}(1)$
Prochain parcours auquel proposer	tableau <code>next_prop[i]</code>	$\mathcal{O}(1)$
Rang de i chez j	dictionnaires <code>rankSpe[j][i]</code> pré-calculés	$\mathcal{O}(1)$
Étudiant le moins préféré chez j	liste <code>aff[j] + rankSpe</code>	$\mathcal{O}(c_j)$
Remplacer un étudiant chez j	<code>aff[j].remove + .append</code>	$\mathcal{O}(c_j)$

Les tables **rankSpe** et **rankEtu** sont construites en $\mathcal{O}(nm)$ et occupent $\mathcal{O}(nm)$ de mémoire. Une variante théoriquement optimale stocke les affectations **aff**[j] dans un tas ordonné par rang, ramenant les deux dernières opérations à $\mathcal{O}(\log c_j)$; en pratique les capacités c_j sont très petites (2 ou 1 dans nos données), nous gardons la liste pour la lisibilité.

Q3 – Gale–Shapley côté étudiants

Algorithme. On gère une file **libre** d'étudiants sans affectation. À chaque étape on extrait i , on consulte $j \leftarrow C_E[i][\text{next_prop}[i]]$ puis :

- si j a une place libre, on accepte i ;
- sinon, on compare i au pire affecté de j : si i est mieux classé, on remplace, l'évincé retournant dans **libre**, sinon i y retourne.

On itère tant que **libre** contient des étudiants ayant encore des parcours à essayer.

Complexité. Chaque étudiant fait au plus m propositions ; chaque proposition comporte un **max** sur les c_j étudiants tenus par j ($c_j \leq n$). On obtient au pire

$$\mathcal{O}(nm c_{\max}) \subseteq \mathcal{O}(n^2 m).$$

Avec un tas par parcours on obtient $\mathcal{O}(nm \log n)$.

Q4 – Gale–Shapley côté parcours

Adaptation. Cette fois ce sont les parcours qui proposent. Chaque parcours j peut faire plusieurs propositions tant qu'il a une place libre. On maintient :

- **next_prop**[j] : prochaine position à essayer dans $C_P[j]$;
- **tenu_par**[i] : indice du parcours qui détient i (**None** sinon) ;
- **places_dispo**[j] : capacité résiduelle de j ;
- file **actifs** des parcours ayant encore une place et un candidat à essayer.

Un parcours j envoie $i \leftarrow C_P[j][\text{next_prop}[j]]$. Si i est libre, il est accepté ; s'il est tenu par j' , on compare les rangs de j et j' chez i via **rankEtu**, et le perdant rejoint la file s'il a encore une place.

Complexité. Symétrique : $\mathcal{O}(nm \log n)$ avec un tas, $\mathcal{O}(n^2 m)$ avec liste. Le nombre total de propositions est borné par nm .

Q5 – Application aux fichiers fournis

Algorithme	Itérations	Stabilité (Q6)
GS côté étudiants	28	stable (0 paire instable)
GS côté parcours	57	stable (0 paire instable)

Sur cette instance, les deux algorithmes renvoient *la même affectation* (le treillis des affectations stables est réduit à un seul élément). Le détail figure dans le tableau de Q15.

Q6 – Vérification de la stabilité

Une paire (i, j) est dite *instable* pour une affectation a si

$$\text{rang}_E[i, j] < \text{rang}_E[i, a(i)] \quad \text{et} \quad (|a^{-1}(j)| < \text{cap}_j \quad \text{ou} \quad \exists i' \in a^{-1}(j), \text{rang}_P[j, i] < \text{rang}_P[j, i']).$$

La fonction **pairesInstables** parcourt les couples (i, j) et utilise **rankEtu**/**rankSpe** pour des tests $\mathcal{O}(1)$, soit $\mathcal{O}(nm + nm c_{\max}) = \mathcal{O}(n^2 m)$ au pire. Les deux affectations renvoyées par GS sont stables, comme garanti par le théorème de Gale–Shapley adapté au problème des hôpitaux.

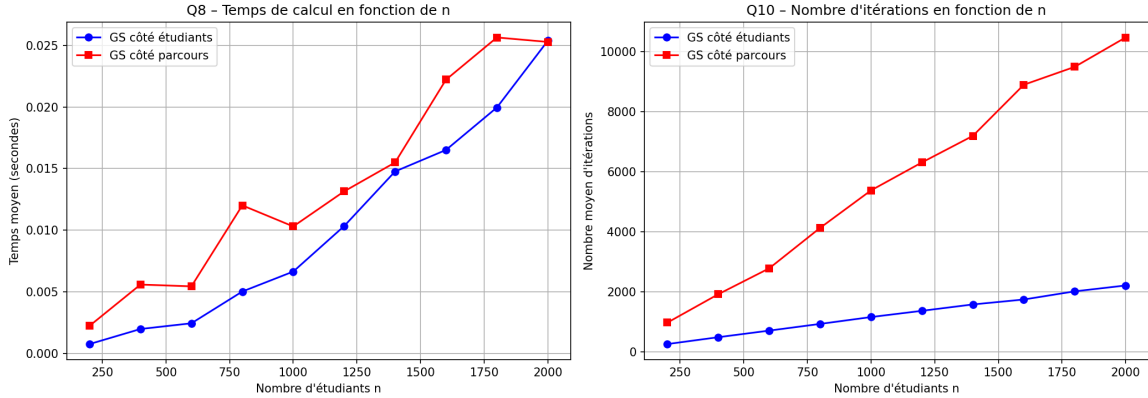
2 Évolution du temps de calcul (Q7–Q10)

Q7 – Génération aléatoire

Les fonctions `genPrefsEtu( $n, m$ )` et `genPrefsSpe( $n, m$ )` génèrent une permutation aléatoire des m parcours pour chaque étudiant (resp. des n étudiants pour chaque parcours), via `random.sample`. Les capacités sont rendues équilibrées et leur somme vaut n pour permettre un mariage parfait.

Q8 – Mesure du temps

Pour $n \in \{200, 400, \dots, 2000\}$, on tire 10 instances aléatoires et on moyenne le temps d'exécution.



n	t_{moy} GS étu (s)	itér. étu	t_{moy} GS spé (s)	itér. spé
200	0,0008	257	0,0023	971
400	0,0020	486	0,0056	1 917
600	0,0025	705	0,0055	2 779
800	0,0050	928	0,0120	4 126
1000	0,0067	1 157	0,0103	5 375
1200	0,0103	1 364	0,0132	6 306
1400	0,0148	1 576	0,0155	7 190
1600	0,0165	1 741	0,0222	8 892
1800	0,0200	2 013	0,0256	9 489
2000	0,0254	2 208	0,0253	10 467

Q9 – Analyse de complexité

Théorique. GS côté étudiants effectue au plus nm propositions et chaque proposition est en $\mathcal{O}(c_{\max})$: avec $m = 10$ fixé et $c_{\max} \approx n/m$ pour des capacités équilibrées, le coût d'une proposition est en $\mathcal{O}(n/m)$, et le coût total est de l'ordre de $\mathcal{O}(n^2)$ au pire.

Empirique. Le temps mesuré croît de 0,8 ms à 25,4 ms quand n passe de 200 à 2000, soit un facteur 32 pour un facteur 10 sur n . Cela correspond à une croissance proche de $n^{1,5}$, comparable à n^2 amorti par les capacités équilibrées (l'opération coûteuse, le `max`, porte sur des listes de taille $\sim n/m$, mais elle n'est invoquée qu'une fraction du temps).

GS côté parcours effectue beaucoup plus d'itérations (10 467 contre 2 208 pour $n = 2000$) car chaque parcours peut redevenir actif lorsqu'un de ses étudiants l'abandonne. Le temps mesuré reste du même ordre grâce à des opérations $\mathcal{O}(1)$ par itération (consultation de `rankEtu`). L'ensemble est conforme à la complexité théorique en $\mathcal{O}(nm \log n)$ avec tas, ou $\mathcal{O}(n^2)$ sans.

Q10 – Nombre d’itérations

Le nombre d’itérations croît *linéairement* en n pour les deux versions (cf. courbe de droite). C’est cohérent avec la borne théorique de nm propositions et la forme amortie de l’algorithme : pour des préférences aléatoires, l’espérance du nombre de propositions est en $\Theta(n \log n)$ pour le mariage stable classique, ce qui est compatible avec la pente quasi-linéaire observée.

3 Équité et PLNE (Q11–Q15)

Q11 – PLNE max-min de l’utilité étudiante

Variables. $x_{ij} \in \{0, 1\}$ pour $i \in \llbracket 1, n \rrbracket, j \in \llbracket 1, m \rrbracket$: $x_{ij} = 1$ si i est affecté à j . $U \in \mathbb{Z}_+$ représente l’utilité minimale.

Modèle.

$$\begin{aligned} & \max U \\ \text{s.c. } & \sum_{j=1}^m x_{ij} = 1 & \forall i \in \llbracket 1, n \rrbracket & \quad (\text{chaque étudiant affecté}) \\ & \sum_{i=1}^n x_{ij} \leq \text{cap}_j & \forall j \in \llbracket 1, m \rrbracket & \quad (\text{capacités}) \\ & U \leq \sum_{j=1}^m u_E[i, j] x_{ij} & \forall i \in \llbracket 1, n \rrbracket & \quad (\text{min des utilités}) \\ & x_{ij} \in \{0, 1\}, U \in \mathbb{Z}_+ \end{aligned}$$

On force le mariage parfait car $\sum_j \text{cap}_j = n$ et un étudiant non affecté aurait une utilité nulle, ce qui fixerait artificiellement $U = 0$.

Résultat (Gurobi). $U^* = 6$, contre $U = 5$ pour les deux solutions Gale–Shapley. Le gain d’équité s’accompagne d’une perte de stabilité : 3 paires deviennent instables.

Q12 – PLNE maximisant la somme des utilités

Modèle.

$$\begin{aligned} & \max \sum_{i=1}^n \sum_{j=1}^m (u_E[i, j] + u_P[j, i]) x_{ij} \\ \text{s.c. } & \sum_{j=1}^m x_{ij} \leq 1 & \forall i \\ & \sum_{i=1}^n x_{ij} \leq \text{cap}_j & \forall j \\ & x_{ij} \in \{0, 1\} \end{aligned}$$

Résultat. La solution optimale fournie par Gurobi vaut $\sum (u_E + u_P) = 239$ pour les 13 étudiants affectés. L’utilité moyenne (*étudiants seuls*) vaut $\overline{u_E} = 8,15$, et l’utilité minimale des étudiants tombe à $u_E^{\min} = 1$: un étudiant est affecté à son dernier choix. Le maximum de la somme des utilités sacrifie totalement l’équité.

Q13 – PLNE maximisant la somme avec contrainte des k premiers choix

L’indication donnée par l’énoncé signifie que la condition « i est dans un de ses k premiers choix » s’écrit linéairement : on annule simplement les variables x_{ij} pour lesquelles $\text{rang}_E[i, j] \geq k$.

Modèle.

$$\begin{array}{ll} \max & \sum_{i,j} (u_E[i,j] + u_P[j,i]) x_{ij} \\ \text{s.c.} & \sum_{j=1}^m x_{ij} \leq 1 \quad \forall i \\ & \sum_{i=1}^n x_{ij} \leq \text{cap}_j \quad \forall j \\ & x_{ij} = 0 \quad \text{si } \text{rang}_E[i,j] \geq k \\ & x_{ij} \in \{0, 1\} \end{array}$$

Q14 – Plus petit k pour un mariage parfait

On résout itérativement le PLNE de Q13 avec la contrainte supplémentaire $\sum_j x_{ij} = 1$ ($\forall i$), pour $k = 1, 2, \dots$. Pour $k \leq 4$ le problème est **infaisable** : il n'existe aucune affectation parfaite dans laquelle chaque étudiant obtient un de ses 4 premiers choix (les goulots viennent essentiellement des parcours MIND, QI et STL, demandés en tête par plusieurs étudiants). Le plus petit k satisfaisant la contrainte est :

$$k^* = 5.$$

L'affectation correspondante est résumée plus loin dans le tableau de comparaison.

Q15 – Comparaison globale des cinq solutions

Solution	$\overline{u_E}$	$u_E^{\min}$	$\overline{u_E + u_P}$	# affectés	# paires inst.	stable ?
GS côté étudiants	8,85	5	17,62	13	0	oui
GS côté parcours	8,85	5	17,62	13	0	oui
PLNE Q11 (max-min)	9,23	6	16,77	13	3	non
PLNE Q12 (max somme)	8,15	1	18,38	13	8	non
PLNE Q14 ($k^* = 5$, parfait)	8,85	6	17,46	13	5	non

Lecture.

- Les deux versions de Gale–Shapley fournissent l'unique affectation stable et atteignent une utilité moyenne très proche de l'optimum social, avec $u_E^{\min} = 5$.
- Le PLNE Q11 améliore strictement le pire étudiant ($u^{\min} : 5 \rightarrow 6$) et même la moyenne, au prix de 3 paires instables et d'une utilité totale moindre.
- Le PLNE Q12 sacrifie complètement l'équité : l'utilité totale est maximale, mais un étudiant est envoyé à son dernier choix et 8 paires sont instables.
- Q14 propose un compromis raisonnable : $u_E^{\min} = 6$ comme Q11, utilité moyenne $\overline{u_E} = 8,85$ comme GS, et 5 paires instables.

Conclusion. Aucune solution ne domine sur tous les critères. Si la stabilité est primordiale, l'algorithme de Gale–Shapley reste le bon choix ; pour favoriser le pire étudiant tout en gardant une bonne moyenne, le PLNE Q11 est préférable ; et pour garantir à chacun *l'un de ses premiers choix*, le PLNE Q14 (avec $k = 5$) constitue un excellent intermédiaire.

Annexe : organisation du code

Tous les fichiers se trouvent dans `new/`.

- `exemple.py` – lecture des fichiers, structures `rankSpe/rankEtu`, `GSEtu`, `GSSpe`, `pairesInstables`, génération aléatoire et mesure du temps.
- `plne.py` – formulations Gurobi des Q11, Q12, Q13 et recherche du plus petit k (Q14).
- `main.py` – exécute Q3 à Q10 sur les fichiers fournis et produit la figure `courbes_GS.png`.
- `comparaison.py` – exécute les cinq solutions et produit le tableau de Q15.